AR arca-sdk

La SDK de ARCA (ex-AFIP) que querías que alguien hiciera.

TypeScript nativo · API limpia en inglés · Tokens automáticos · QR oficial · Padrón A13

npm v1.1.0 | downloads 2.6k/month | license MIT | node >=18 | TypeScript 5.x

---

¿Por qué arca-sdk?

La mayoría de las librerías de AFIP/ARCA para Node.js son:

- Ports de código PHP/Java sin tipos
- Sin soporte para el Padrón A13
- Sin manejo de tokens (te obligan a gestionarlos a mano)
- Sin QR oficial
- Sin mantenimiento activo

`arca-sdk` fue construida desde cero en TypeScript moderno, probada contra producción real, y documenta quirks del spec oficial que ninguna otra librería menciona (como el `encodeURIComponent` que rompe el scanner de ARCA).

Instalación

```
# npm
npm install arca-sdk

# bun
bun add arca-sdk

# pnpm
pnpm add arca-sdk
```

Requisitos: Node.js 18+ · TypeScript 5+ (optional but recommended) · Bun compatible

---

🔐 Security & Responsibility / Seguridad y Responsabilidad

US English

arca-sdk is designed to be **stateless and cloud-native**. It does **NOT** persist certificates or private keys to the filesystem.

It is the responsibility of the implementing application to:

1. **Securely store** the Private Key (`.key`) and Certificate (`.crt`). (Recommended: Encrypted Database, AWS KMS, HashiCorp Vault).
2. **Decrypt** credentials only at runtime.
3. Pass the raw strings/buffers to the SDK constructors.

> *[!WARNING] Never commit your `.key` files to Git or expose them in public folders. The SDK operates in-memory to ensure maximum security in Serverless environments (Vercel, AWS Lambda).*

AR Español

arca-sdk está diseñado para ser **stateless** (sin estado) y **cloud-native**. **NO** guarda certificados ni claves privadas en el sistema de archivos.

Es responsabilidad de la aplicación que implementa el SDK:

1. **Almacenar de forma segura** la Clave Privada (`.key`) y el Certificado (`.crt`). (Recomendado: Base de Datos encriptada, AWS KMS, HashiCorp Vault).
2. **Desencriptar** las credenciales solo en tiempo de ejecución.
3. Pasar los strings o buffers crudos a los constructores del SDK.

> *[!CAUTION] Nunca subas tus archivos `.key` a Git ni los expongas en carpetas públicas. El SDK opera en memoria para garantizar la máxima seguridad en entornos Serverless (Vercel, AWS Lambda).*

Quick Start — 5 minutos y estás facturando

```
import * as fs from 'fs';
import { WsaaService, WsfeService } from 'arca-sdk';

// 1. Autenticación con WSAA (se renueva automáticamente)
const wsaa = new WsaaService({
  environment: 'homologacion', // 'produccion' cuando estés listo
  cuit: '20123456789',
  cert: fs.readFileSync('cert.pem', 'utf-8'),
  key:  fs.readFileSync('key.pem',  'utf-8'),
  service: 'wsfe',
});

const ticket = await wsaa.login();

// 2. Servicio de facturación
const wsfe = new WsfeService({
  environment: 'homologacion',
  cuit: '20123456789',
  ticket,
  pointOfSale: 4,
});

// 3. Emitir Ticket C — una línea
const result = await wsfe.issueSimpleReceipt({ total: 1500 });

console.log('CAE:', result.cae);              // '75157992335329'
console.log('Vto:', result.caeExpiry);        // '20260302'
console.log('QR:', result.qrUrl);             // 'https://www.afip.gob.ar/fe/qr/?p=...'
```

> *Los certificados se obtienen en el [portal de ARCA](#) (CLAVE FISCAL nivel 3+).*

Funcionalidades

✅ Servicios soportados

Servicio	Descripción	Estado
WSAA	Autenticación y Autorización	✅ Completo
WSFE v1	Facturación Electrónica (A, B, C)	✅ Completo
Padrón A13	Consulta de datos de contribuyentes	✅ Completo

✅ Tipos de comprobantes

Método	Comprobante	Cuándo usarlo
`issueSimpleReceipt()`	Ticket C	Monto total, sin detalle. Ideal para POS simple
`issueReceipt()`	Ticket C + items	Con detalle de productos guardado localmente
`issueInvoiceC()`	Factura C	Monotributistas a consumidor final / Empresas
`issueInvoiceB()`	Factura B	Responsable Inscripto a consumidor final / Monotributo
`issueInvoiceA()`	Factura A	Responsable Inscripto a Responsable Inscripto
`issueCreditNoteA/B/C()`	Nota de Crédito	Anulación/Devolución (Requiere asociar la factura original)
`issueDebitNoteA/B/C()`	Nota de Débito	Cobro extra/Penalidad (Requiere asociar la factura original)
`issueReceiptA/B/C()`	Recibo	Comprobante de pago (misma emisión que una factura)

✅ Consultas disponibles

Método	Descripción
`wsfe.getInvoice(type, n)`	Consulta un comprobante ya emitido (FECompConsultar)
`wsfe.getPointsOfSale()`	Lista puntos de venta habilitados (FEParamGetPtosVenta)
`WsfeService.checkStatus()`	Estado de los servidores de ARCA (FEDummy)
`padron.getTaxpayer(cuit)`	Datos del contribuyente — nombre, domicilio, condición IVA

Ejemplos

Ticket C con detalle de items

```
const result = await wsfe.issueReceipt({
  items: [
    { description: 'Café con leche',  quantity: 2, unitPrice: 750 },
    { description: 'Medialunas x4',   quantity: 1, unitPrice: 600 },
  ],
```

```
});

console.log('Items en respuesta:', result.items?.length); // 2
console.log('QR URL:', result.qrUrl);
```

Factura B con IVA discriminado

```
import { TaxIdType } from 'arca-sdk';

const result = await wsfe.issueInvoiceB({
  items: [
    { description: 'Servicio de diseño', quantity: 10, unitPrice: 1000, vatRate: 21 },
    { description: 'Hosting mensual',    quantity: 1,  unitPrice: 5000, vatRate: 21 },
  ],
  buyer: {
    docType: TaxIdType.CUIT,
    docNumber: '20987654321',
  },
});

// VAT breakdown (required by ARCA for A/B invoices)
result.vat?.forEach(v => {
  console.log(`IVA ${v.rate}%: base $${v.taxBase} → $${v.amount}`);
});
```

Nota de Crédito (Anulando factura previa)

```
import { InvoiceType } from 'arca-sdk';

const result = await wsfe.issueCreditNoteC({
  items: [
    { description: 'Anulación de equipo defectuoso', quantity: 1, unitPrice: 45000 },
  ],
  // ⚠️ Obligatorio en NC/ND: especificar el comprobante original afectado
  associatedInvoices: [{
    type: InvoiceType.FACTURA_C, // La factura que estoy anulando
    pointOfSale: 4,
    invoiceNumber: 15302,
  }],
});

console.log('CAE de anulación:', result.cae);
```

Consulta de Padrón A13

```
import { PadronService } from 'arca-sdk';

const padron = new PadronService({
```

```
  environment: 'homologacion',
  cuit: '20123456789',
  cert: fs.readFileSync('cert.pem', 'utf-8'),
  key:  fs.readFileSync('key.pem',  'utf-8'),
});

const { taxpayer, error } = await padron.getTaxpayer('30111111118');
if (taxpayer) {
  const name = taxpayer.companyName || `${taxpayer.firstName} ${taxpayer.lastName}`;
  console.log('Nombre:', name);
  console.log('Provincia:', taxpayer.addresses[0]?.province);
  console.log('¿IVA?:', taxpayer.isVATRegistered);
  console.log('¿Mono?:', taxpayer.isMonotax);
}
```

God Mode: persistencia automática de tokens

Pasale un adaptador `storage` y el SDK gestiona el ciclo de vida del ticket solo — incluyendo renovación automática al expirar.

```
const wsaa = new WsaaService({
  ...config,
  service: 'wsfe',
  storage: {
    // Leé el ticket guardado (de DB, Redis, filesystem, lo que sea)
    get: async (cuit, env) => {
      const row = await db.token.findUnique({ where: { cuit, env } });
      return row ? { token: row.token, sign: row.sign, ... } : null;
    },
    // Guardá el ticket nuevo
    save: async (cuit, env, ticket) => {
      await db.token.upsert({ ... });
    },
  }
});

// Desde ahora, .login() va a buscar el token en la DB antes de pedirle uno a ARCA
const ticket = await wsaa.login();
```

QR oficial de ARCA

```
import { generateQRUrl } from 'arca-sdk';

// 1. Ya viene integrado en todos los métodos de emisión:
const result = await wsfe.issueSimpleReceipt({ total: 1500 });
console.log(result.qrUrl); // listo para embeber en un generador de QR

// 2. O generalo a mano si ya tenés la respuesta:
const url = generateQRUrl(caeResponse, '20123456789', 1500.00);
```

> ***Nota:*** *La URL usa base64 crudo sin* `encodeURIComponent`*. Es un quirk documentado del spec de ARCA — su scanner no acepta caracteres URL-encoded.*

Manejo de errores

Todos los errores son instancias tipadas de `ArcaError`, con un campo `hint` que te dice qué hacer:

```
import { ArcaError, ArcaAuthError, ArcaValidationError, ArcaNetworkError } from 'arca-sdk';

try {
  const result = await wsfe.issueSimpleReceipt({ total: 1500 });
} catch (error) {
  if (error instanceof ArcaAuthError) {
    // Token expirado, certificado inválido, etc.
    console.error('Auth error:', error.message);
    console.log('Hint:', error.hint); // → "El certificado puede haber expirado..."
  } else if (error instanceof ArcaValidationError) {
    // Datos inválidos antes de llamar a ARCA
    console.error('Validation:', error.message, error.details);
  } else if (error instanceof ArcaNetworkError) {
    // Timeout, error HTTP
    console.error('Network:', error.message);
  } else if (error instanceof ArcaError) {
    // Error semántico de ARCA (código de error en la respuesta SOAP)
    console.error(`ARCA Error [${error.code}]:`, error.message);
    console.log('Hint:', error.hint); // Pista específica por código de error
  }
}
```

🚚 Acerca de los Remitos

> ***¡Atención!*** *Este SDK implementa nativamente el servicio* `WSFE` *(Facturación Electrónica). Si tu negocio necesita emitir* ***Remitos Electrónicos Oficiales*** *para el traslado físico de mercaderías (Remitos Cárnicos, Azucareros, Harineros, etc.), tené en cuenta que la AFIP exige usar un webservice totalmente distinto llamado* `WSREM` *o similares. Estos servicios aún no están cubiertos por esta versión del SDK.*

Compatibilidad

Runtime	Versión mínima	Estado
Node.js	18 LTS	✅ Soportado
Node.js	20 LTS	✅ Soportado
Node.js	22 LTS	✅ Soportado
Bun	1.x	✅ Soportado
Deno	—	⚠️ Sin probar
Browser	—	❌ No soportado (requiere `node:https`)

> *El SDK maneja automáticamente los errores SSL de los servidores de ARCA ("dh key too small") mediante configuración custom del* `https.Agent`.

Tipos exportados

```
// Servicios
import { WsaaService, WsfeService, PadronService } from 'arca-sdk';

// Enums
import { InvoiceType, BillingConcept, TaxIdType } from 'arca-sdk';

// Tipos de configuración
import type { WsaaConfig, WsfeConfig, TaxpayerServiceConfig } from 'arca-sdk';

// Tipos de respuesta
import type { CAEResponse, InvoiceDetails, PointOfSale, ServiceStatus } from 'arca-sdk';
import type { TaxpayerResponse, Taxpayer, Address, Activity, TaxRecord } from 'arca-sdk';

// Items de factura
import type { InvoiceItem, Buyer, IssueInvoiceRequest } from 'arca-sdk';

// Storage
import type { TokenStorage, LoginTicket } from 'arca-sdk';

// Errores
import { ArcaError, ArcaAuthError, ArcaValidationError, ArcaNetworkError } from 'arca-sdk';

// QR
import { generateQRUrl } from 'arca-sdk';
```

Desarrollo

```
# Clonar e instalar
git clone https://github.com/marcelaborgarello/arca-sdk
cd arca-sdk
bun install

# Tests
bun test

# Verificar tipos
bun run lint

# Build (CJS + ESM + .d.ts)
bun run build
```

Tests disponibles

Suite	Archivo	Qué cubre
Cálculos	`calculations.test.ts`	IVA, subtotales, totales con y sin IVA incluido
QR	`qr.test.ts`	Generación de URL, limpieza de CUIT/CAE, campo comprador
XML	`xml.test.ts`	Construcción de TRA, parsing WSAA, validación de CUIT
Padrón	`padron.test.ts`	Parsing de respuesta, CUIT not found, condición IVA
WSFE	`wsfe.test.ts`	issueSimpleReceipt, issueReceipt, issueInvoiceB, checkStatus

---

Roadmap

- [ ] Soporte WSMTXCA (Factura de Crédito Electrónica MiPyME)
- [ ] Soporte WSCT (Turismo)
- [ ] Método `consultar()` para servicios adicionales del Padrón
- [ ] Opción de exportar a PDF (recibo y factura)

Licencia

MIT © Marcela Borgarello

Hecho con ❤️ en Argentina AR

Porque integrar con ARCA no tiene por qué ser un infierno.

npm · GitHub · CHANGELOG